

Polychronous Multi-Modal Spiking Memory

Unified implementation fixing the prototype gaps

This notebook adds the missing neural layers and the end-to-end multimodal memory flow from the specification.

In [1]: `%pip install numpy matplotlib`

```
import numpy as np
from typing import Dict, List, Tuple, Set, Optional
from dataclasses import dataclass
from collections import defaultdict
import hashlib
import random

print("✅ Environment ready")
```

Requirement already satisfied: numpy in ./venv/lib/python3.12/site-packages (2.5.1)

Requirement already satisfied: matplotlib in ./venv/lib/python3.12/site-packages (3.11.1)

Requirement already satisfied: contourpy>=1.0.1 in ./venv/lib/python3.12/site-packages (from matplotlib) (1.3.3)

Requirement already satisfied: cyclor>=0.10 in ./venv/lib/python3.12/site-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.28.2 in ./venv/lib/python3.12/site-packages (from matplotlib) (4.63.0)

Requirement already satisfied: kiwisolver>=1.3.1 in ./venv/lib/python3.12/site-packages (from matplotlib) (1.5.0)

Requirement already satisfied: packaging>=20.0 in ./venv/lib/python3.12/site-packages (from matplotlib) (26.3)

Requirement already satisfied: pillow>=9 in ./venv/lib/python3.12/site-packages (from matplotlib) (12.3.0)

Requirement already satisfied: pyparsing>=3 in ./venv/lib/python3.12/site-packages (from matplotlib) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in ./venv/lib/python3.12/site-packages (from matplotlib) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in ./venv/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

Note: you may need to restart the kernel to use updated packages.

✅ Environment ready

In [2]:

```
"""
Polychronous Multi-Modal Spiking Memory
Unified implementation fixing all prototype gaps.
"""

import numpy as np
from typing import Dict, List, Tuple, Set, Optional
from dataclasses import dataclass, field
```

```

from collections import defaultdict
import hashlib
import random

# =====
# 1. NEURAL PRIMITIVES
# =====

@dataclass
class LIFNeuron:
    """Leaky Integrate-and-Fire neuron."""
    nid: int
    tau_m: float = 20.0      # ms
    v_rest: float = -70.0    # mV
    v_thresh: float = -55.0  # mV
    v_reset: float = -75.0   # mV
    tau_ref: float = 2.0     # ms
    dt: float = 0.1         # ms

    def __post_init__(self):
        self.v = self.v_rest
        self.refractory_until = -1.0
        self.spike_times: List[float] = []
        self._last_I = 0.0

    def step(self, t: float, I_syn: float) -> bool:
        """Euler step. Returns True if spike emitted."""
        if t < self.refractory_until:
            self.v = self.v_reset
            return False

        dv = (-(self.v - self.v_rest) + I_syn) / self.tau_m * self.dt
        self.v += dv

        if self.v >= self.v_thresh:
            self.v = self.v_reset
            self.refractory_until = t + self.tau_ref
            self.spike_times.append(t)
            return True
        return False

    def reset(self):
        self.v = self.v_rest
        self.refractory_until = -1.0
        self.spike_times.clear()

# =====
# 2. SYNAPSE MODEL
# =====

class DelayedSynapse:
    """Delayed synapse with exponential PSC and STDP."""

    def __init__(self, pre: str, post: int, weight: float, delay: float,
                 tau_syn: float = 3.0, w_max: float = 10.0, w_min: float = 0.0):

```

```

self.pre = pre          # Pre-synaptic neuron ID (string for input,
self.post = post        # Post-synaptic neuron index
self.w = weight
self.delay = delay      # ms
self.tau_syn = tau_syn
self.w_max = w_max
self.w_min = w_min

# STDP parameters
self.A_plus = 0.5
self.A_minus = 0.48
self.tau_plus = 12.0
self.tau_minus = 12.0

# Spike tracking for PSC
self.pre_spike_times: List[float] = []

def current(self, t: float) -> float:
    """Compute postsynaptic current at time t."""
    I = 0.0
    for t_pre in self.pre_spike_times:
        s = t - t_pre - self.delay
        if s >= 0:
            I += self.w * (s / self.tau_syn) * np.exp(-s / self.tau_syn)
    return I

def add_spike(self, t: float):
    self.pre_spike_times.append(t)

def stdp_update(self, t_post: float, pre_spike_times: List[float], M: fl
    """
    STDP update when post-synaptic neuron fires.
    M: neuromodulatory gain (0=no plasticity, 1=normal, 2=boosted)
    """
    if M == 0:
        return
    for t_pre in pre_spike_times:
        dt = t_post - (t_pre + self.delay)
        if dt >= 0:
            dw = M * self.A_plus * np.exp(-dt / self.tau_plus)
        else:
            dw = -M * self.A_minus * np.exp(dt / self.tau_minus)
        self.w = np.clip(self.w + dw, self.w_min, self.w_max)

def reset(self):
    self.pre_spike_times.clear()

# =====
# 3. MODALITY ENCODERS
# =====

class SQLEncoder:
    """100 neurons: 5 fields × 20 population code."""

    FIELDS = ['age', 'salary', 'city', 'role', 'dept']

```

```

N_PER_FIELD = 20
TOTAL = 100

def __init__(self, duration: float = 10.0, dt: float = 0.1):
    self.duration = duration
    self.dt = dt
    self.centers = {
        'age': np.linspace(20, 80, self.N_PER_FIELD),
        'salary': np.linspace(0, 150000, self.N_PER_FIELD),
    }
    self.sigma = {'age': 10.0, 'salary': 20000.0}
    self._cat_cache: Dict[str, int] = {}

def _cat_neuron(self, field: str, value: str) -> int:
    """Hash categorical value to consistent neuron within field."""
    key = f"{field}:{value}"
    if key not in self._cat_cache:
        h = int(hashlib.md5(key.encode()).hexdigest(), 16)
        self._cat_cache[key] = h % self.N_PER_FIELD
    return self._cat_cache[key]

def encode(self, row: Dict) -> Dict[int, List[float]]:
    """Return {neuron_id: [spike_times]}."""
    spikes = defaultdict(list)
    base = 0
    for field in self.FIELDS:
        val = row.get(field, 0)
        if field in self.centers:
            # Numeric: population rate coding
            for i, c in enumerate(self.centers[field]):
                rate = 100.0 * np.exp(-(val - c)**2 / (2 * self.sigma[field]))
                if rate > 5.0:
                    n_spikes = np.random.poisson(rate * self.duration / self.dt)
                    for _ in range(n_spikes):
                        t = np.random.uniform(0, self.duration)
                        spikes[base + i].append(t)
            else:
                # Categorical: WTA single spike
                idx = self._cat_neuron(field, str(val))
                spikes[base + idx].append(0.0)
        base += self.N_PER_FIELD
    return dict(spikes)

def decode_numeric(self, spikes: Dict[int, List[float]], field: str) -> float:
    """Decode numeric field from spike counts."""
    if field not in self.centers:
        return 0.0
    base = self.FIELDS.index(field) * self.N_PER_FIELD
    counts = np.zeros(self.N_PER_FIELD)
    for nid, times in spikes.items():
        if base <= nid < base + self.N_PER_FIELD:
            counts[nid - base] = len(times)
    if counts.sum() == 0:
        return 0.0
    return np.average(self.centers[field], weights=counts + 1e-6)

```

```

def decode_category(self, spikes: Dict[int, List[float]], field: str) ->
    """Decode categorical field (best effort)."""
    base = self.FIELDS.index(field) * self.N_PER_FIELD
    best = None
    best_count = -1
    for nid, times in spikes.items():
        if base <= nid < base + self.N_PER_FIELD:
            if len(times) > best_count:
                best_count = len(times)
                best = nid - base
    # Reverse lookup
    for k, v in self._cat_cache.items():
        if k.startswith(f"{field}:") and v == best:
            return k.split(":", 1)[1]
    return ""

class GraphEncoder:
    """80 neurons: 4 nodes × 20 ensemble code, delay-coded edges."""

    N_NODES = 4
    N_PER_NODE = 20
    K_ACTIVE = 5
    TOTAL = 80

    RELATION_DELAYS = {
        'WORKS_AT': 3.0,
        'FRIENDS_WITH': 7.0,
        'MANAGES': 5.0,
        'REPORTS_TO': 2.0,
    }

    def __init__(self, duration: float = 8.0, dt: float = 0.1, seed: int = 4):
        self.duration = duration
        self.dt = dt
        rng = np.random.RandomState(seed)
        # Fixed random binary codes for each node
        self.node_codes = {
            i: set(rng.choice(self.N_PER_NODE, self.K_ACTIVE, replace=False))
            for i in range(self.N_NODES)
        }

    def encode(self, edge: Tuple[int, str, int]) -> Dict[int, List[float]]:
        """Edge as (node_a, relation, node_b). Returns spike dict."""
        src, rel, tgt = edge
        delay = self.RELATION_DELAYS.get(rel, 3.0)
        spikes = defaultdict(list)
        t_base = 1.0 # ms offset

        # Source node fires at t_base
        for i in self.node_codes[src]:
            spikes[i].append(t_base)

        # Target node fires at t_base + delay
        offset = self.N_PER_NODE
        for i in self.node_codes[tgt]:

```

```

        spikes[offset + i].append(t_base + delay)

    return dict(spikes)

def get_active_neurons(self, node_id: int, side: str = 'source') -> Set[
    """Get neuron IDs for a node."""
    if side == 'source':
        return {i for i in self.node_codes[node_id]}
    else:
        return {self.N_PER_NODE + i for i in self.node_codes[node_id]}

class TextEncoder:
    """100 neurons: spiking tokenizer with phase-of-firing."""

    TOTAL = 100
    SPARSITY = 0.05 # 5% active = 5 neurons per word

    def __init__(self, duration_per_word: float = 5.0, gamma_cycle: float =
        self.duration_per_word = duration_per_word
        self.gamma_cycle = gamma_cycle
        rng = np.random.RandomState(seed)
        # Fixed random word codes
        self.word_codes: Dict[str, Set[int]] = {}
        self.rng = rng

    def _get_code(self, word: str) -> Set[int]:
        if word not in self.word_codes:
            self.word_codes[word] = set(self.rng.choice(self.TOTAL, int(self
        return self.word_codes[word]

    def encode(self, sentence: str) -> Dict[int, List[float]]:
        """Encode sentence as phase-coded word bursts."""
        words = sentence.lower().split()
        spikes = defaultdict(list)
        for pos, word in enumerate(words):
            phase = (pos % 4) * (self.gamma_cycle / 4) # 4 phases per cycle
            code = self._get_code(word)
            for nid in code:
                spikes[nid].append(phase + 1.0) # +1ms offset
        return dict(spikes)

# =====
# 4. LAYER 2: ENTORHINAL CONVERGENCE (EC)
# =====

class EntorhinalCortex:
    """180 neurons. Computes novelty energy and triggers ACh."""

    def __init__(self, n_sql: int = 100, n_graph: int = 80, n_text: int = 10
        theta_novelty: float = 2.0, history_window: int = 10):
        self.n_sql = n_sql
        self.n_graph = n_graph
        self.n_text = n_text
        self.dim = n_sql + n_graph + n_text # 280 with text, 180 without

```

```

self.theta_novelty = theta_novelty
self.history_window = history_window

# Running statistics for novelty
self.energy_history: List[float] = []
self.mean_energy = 0.0
self.std_energy = 1.0

def _spikes_to_rate_vector(self, spikes: Dict[int, List[float]], dim: int) -> np.ndarray:
    x = np.zeros(dim)
    for nid, times in spikes.items():
        if 0 <= nid < dim:
            x[nid] = len(times) / (duration / 1000.0) # Hz
    return x

def compute_novelty(self, sql_spikes: Dict[int, List[float]],
                    graph_spikes: Dict[int, List[float]],
                    text_spikes: Optional[Dict[int, List[float]]] = None,
                    duration: float = 10.0) -> Tuple[float, int]:
    """
    Returns (energy, M) where M is neuromodulatory state:
    M=2 (novel/ACh boost) if energy > mean + theta * std
    M=1 (familiar) otherwise
    """
    x_sql = self._spikes_to_rate_vector(sql_spikes, self.n_sql, duration)
    x_graph = self._spikes_to_rate_vector(graph_spikes, self.n_graph, duration)

    if text_spikes is not None:
        x_text = self._spikes_to_rate_vector(text_spikes, self.n_text, duration)
        x = np.concatenate([x_sql, x_graph, x_text])
    else:
        x = np.concatenate([x_sql, x_graph])

    energy = np.linalg.norm(x) ** 2

    # Update running stats
    self.energy_history.append(energy)
    if len(self.energy_history) > self.history_window:
        self.energy_history.pop(0)
    self.mean_energy = np.mean(self.energy_history)
    self.std_energy = np.std(self.energy_history) + 1e-6

    deviation = (energy - self.mean_energy) / self.std_energy
    M = 2 if deviation > self.theta_novelty else 1

    return energy, M

# =====
# 5. LAYER 3: DENTATE GYRUS (DG)
# =====

class DentateGyrus:
    """900 neurons. Sparse separator via random projection + k-WTA."""

    def __init__(self, input_dim: int = 180, output_dim: int = 900,

```

```

        target_sparsity: float = 0.03, seed: int = 123):
    self.input_dim = input_dim
    self.output_dim = output_dim
    self.k = max(1, int(output_dim * target_sparsity)) # ~27 active
    rng = np.random.RandomState(seed)
    # Random dictionary (could be learned via sparse coding in Phase 2)
    self.W = rng.randn(output_dim, input_dim) * 0.1

    def encode(self, x: np.ndarray) -> Set[int]:
        """Project input to sparse code."""
        h = self.W @ x
        # k-Winner-Take-All
        top_k = np.argsort(h)[-self.k:]
        return set(top_k)

    def to_spikes(self, active: Set[int], t_offset: float = 0.0) -> Dict[int, List[float]]:
        """Convert active set to synchronous spike dict."""
        return {nid: [t_offset + np.random.uniform(0, 1.0)] for nid in active}

# =====
# 6. LAYER 4: CA3 RECURRENT ATTRACTOR
# =====

class CA3Attractor:
    """
    300 neurons: 240 RS excitatory + 60 FS inhibitory.
    Recurrent connectivity with STDP.
    """

    def __init__(self, n_e: int = 240, n_i: int = 60, dt: float = 0.1, seed: int = 123):
        self.n_e = n_e
        self.n_i = n_i
        self.N = n_e + n_i
        self.dt = dt
        self.rng = np.random.RandomState(seed)

        # Create neurons
        self.neurons: List[LIFNeuron] = []
        for i in range(n_e):
            self.neurons.append(LIFNeuron(nid=i, tau_m=20.0, v_thresh=-55.0,
                                           v_reset=-75.0, tau_ref=2.0, dt=dt))
        for i in range(n_i):
            self.neurons.append(LIFNeuron(nid=n_e+i, tau_m=15.0, v_thresh=-55.0,
                                           v_reset=-75.0, tau_ref=1.0, dt=dt))

        # Synapses
        self.synapses: List[DelayedSynapse] = []
        self.incoming: Dict[int, List[DelayedSynapse]] = defaultdict(list)
        self.outgoing: Dict[int, List[DelayedSynapse]] = defaultdict(list)

        # Recurrent connectivity
        self._connect_recurrent()

        # Input synapses (populated externally)
        # Key: synapse.pre (string ID), Value: synapse

```



```

self.input_synapses: Dict[str, DelayedSynapse] = {}

# State tracking for continuous simulation
self.time = 0.0

def _connect_recurrent(self):
    """Build recurrent E→E, E→I, I→E connectivity."""
    # E→E (15%)
    for i in range(self.n_e):
        for j in range(self.n_e):
            if i != j and self.rng.random() < 0.15:
                w = self.rng.uniform(0.2, 0.4)
                d = self.rng.uniform(1.0, 3.0)
                syn = DelayedSynapse(pre=str(i), post=j, weight=w, delay=d)
                self.synapses.append(syn)
                self.incoming[j].append(syn)
                self.outgoing[i].append(syn)

    # E→I (20%)
    for i in range(self.n_e):
        for j in range(self.n_i):
            if self.rng.random() < 0.20:
                w = self.rng.uniform(0.3, 0.5)
                d = self.rng.uniform(0.5, 2.0)
                syn = DelayedSynapse(pre=str(i), post=self.n_e+j, weight=w, delay=d)
                self.synapses.append(syn)
                self.incoming[self.n_e+j].append(syn)
                self.outgoing[i].append(syn)

    # I→E (30%)
    for i in range(self.n_i):
        for j in range(self.n_e):
            if self.rng.random() < 0.30:
                w = self.rng.uniform(-1.2, -0.8)
                d = self.rng.uniform(0.5, 1.5)
                syn = DelayedSynapse(pre=str(self.n_e+i), post=j, weight=w, delay=d)
                self.synapses.append(syn)
                self.incoming[j].append(syn)
                self.outgoing[self.n_e+i].append(syn)

def add_input_synapse(self, pre_id: str, post: int, weight: float, delay: float):
    """Add feedforward input synapse. pre_id must be string matching spike_id
    syn = DelayedSynapse(pre=pre_id, post=post, weight=weight, delay=delay)
    self.synapses.append(syn)
    self.incoming[post].append(syn)
    self.input_synapses[pre_id] = syn

def run(self, duration: float, input_spikes: Dict[str, List[float]],
        M: float = 0.0, record: bool = True) -> Dict[int, List[float]]:
    """
    Run simulation for duration ms.
    input_spikes: {pre_neuron_id: [spike_times]} where pre_id matches syn_pre_id
    M: STDP modulation (0=None, 1=normal, 2=boosted)
    """
    t_start = self.time
    t_end = self.time + duration

```

```

steps = int(duration / self.dt)

# Schedule input spikes
for pre_id, times in input_spikes.items():
    if pre_id in self.input_synapses:
        syn = self.input_synapses[pre_id]
        for t_spike in times:
            if t_start <= t_spike < t_end:
                syn.add_spike(t_spike)

# Simulation loop
for step in range(steps):
    t = t_start + step * self.dt

    # 1. Compute synaptic currents
    I_syn = np.zeros(self.N)
    for syn in self.synapses:
        I_syn[syn.post] += syn.current(t)

    # 2. Step neurons
    for i, neuron in enumerate(self.neurons):
        spiked = neuron.step(t, I_syn[i])
        if spiked:
            # Propagate to outgoing synapses
            nid_str = str(i)
            for syn in self.outgoing[i]:
                syn.add_spike(t)
            # STDP on incoming synapses
            for syn in self.incoming[i]:
                # Get pre neuron spike times
                pre_nid = int(syn.pre) if syn.pre.isdigit() else None
                if pre_nid is not None and 0 <= pre_nid < self.N:
                    pre_spikes = self.neurons[pre_nid].spike_times
                else:
                    # Input synapse: use scheduled spikes
                    pre_spikes = syn.pre_spike_times
                syn.stdp_update(t, pre_spikes, M)

self.time = t_end

if record:
    return {i: n.spike_times.copy() for i, n in enumerate(self.neurons)}
return {}

def get_active_neurons(self, threshold: int = 1) -> Set[int]:
    """Return set of neuron IDs that fired >= threshold spikes."""
    return {i for i, n in enumerate(self.neurons) if len(n.spike_times) >= threshold}

def reset(self):
    """Reset neural state (use only for fresh episodes, not between stages)"""
    for n in self.neurons:
        n.reset()
    for syn in self.synapses:
        syn.reset()
    self.time = 0.0

```

```

def soft_reset(self):
    """Clear spike times but preserve membrane potentials (for continuous)
    for n in self.neurons:
        n.spike_times.clear()
    for syn in self.synapses:
        syn.pre_spike_times.clear()

# =====
# 7. LAYER 5: CA1 READOUT
# =====

class CA1Readout:
    """180 neurons mapping CA3 back to encoder space."""

    def __init__(self, n_ca3: int = 240, n_ca1: int = 180, seed: int = 77):
        self.n_ca3 = n_ca3
        self.n_ca1 = n_ca1
        rng = np.random.RandomState(seed)
        # Random back-projection weights
        self.W = rng.randn(n_ca1, n_ca3) * 0.05
        self.bias = rng.randn(n_ca1) * 0.01

    def decode(self, ca3_spikes: Dict[int, List[float]], duration: float) ->
        """Reconstruct rate vector from CA3 activity."""
        r_ca3 = np.zeros(self.n_ca3)
        for nid, times in ca3_spikes.items():
            if nid < self.n_ca3:
                r_ca3[nid] = len(times) / (duration / 1000.0)
        r_ca1 = self.W @ r_ca3 + self.bias
        return np.maximum(r_ca1, 0) # ReLU

    def decode_sql(self, ca3_spikes: Dict[int, List[float]], duration: float)
        """Decode SQL portion (first 100 dims)."""
        r = self.decode(ca3_spikes, duration)
        return {i: r[i] for i in range(min(100, len(r)))}

    def decode_graph(self, ca3_spikes: Dict[int, List[float]], duration: float)
        """Decode Graph portion (next 80 dims)."""
        r = self.decode(ca3_spikes, duration)
        return {i: r[i] for i in range(100, min(180, len(r)))}

# =====
# 8. MULTI-MODAL MEMORY SYSTEM
# =====

class MultiModalMemory:
    """
    Complete system: Encoders -> EC -> DG -> CA3 -> CA1.
    Implements continuous two-stage encoding protocol.
    """

    def __init__(self, use_text: bool = False, seed: int = 42):
        self.use_text = use_text
        self.sql_enc = SQLEncoder()

```

```

self.graph_enc = GraphEncoder()
self.text_enc = TextEncoder() if use_text else None

# EC: input dim depends on modalities
ec_dim = 280 if use_text else 180
self.ec = EntorhinalCortex(n_text=(100 if use_text else 0))

# DG
self.dg = DentateGyrus(input_dim=ec_dim)

# CA3
self.ca3 = CA3Attractor()

# CA1
self.ca1 = CA1Readout()

# Build input pathways with CORRECT routing
self._build_input_pathways()

# Engram storage
self.engrams: List[Set[int]] = [] # List of active CA3 neuron sets
self.episode_count = 0

def _build_input_pathways(self):
    """Create input synapses with IDs matching spike routing."""
    rng = np.random.RandomState(42)

    # SQL inputs: IDs "0" to "99" → CA3 excitatory neurons
    for sql_nid in range(100):
        pre_id = str(sql_nid) # CRITICAL FIX: matches spike routing
        for ca3_nid in range(self.ca3.n_e):
            if rng.random() < 0.08:
                w = rng.uniform(0.5, 0.6)
                d = rng.uniform(3.0, 6.0)
                self.ca3.add_input_synapse(pre_id, ca3_nid, w, d)

    # Graph inputs: IDs "100" to "179" → CA3 excitatory neurons
    for graph_nid in range(80):
        pre_id = str(100 + graph_nid) # CRITICAL FIX: offset by 100
        for ca3_nid in range(self.ca3.n_e):
            if rng.random() < 0.08:
                w = rng.uniform(0.5, 0.6)
                d = rng.uniform(0.0, 2.0)
                self.ca3.add_input_synapse(pre_id, ca3_nid, w, d)

    # Text inputs: IDs "180" to "279" → CA3 excitatory neurons
    if self.use_text:
        for text_nid in range(100):
            pre_id = str(180 + text_nid)
            for ca3_nid in range(self.ca3.n_e):
                if rng.random() < 0.05:
                    w = rng.uniform(0.4, 0.5)
                    d = rng.uniform(1.0, 4.0)
                    self.ca3.add_input_synapse(pre_id, ca3_nid, w, d)

def _merge_spikes(self, *spike_dicts) -> Dict[str, List[float]]:

```

```

"""Merge spike dicts with proper ID string conversion."""
merged = defaultdict(list)
for spikes in spike_dicts:
    for nid, times in spikes.items():
        merged[str(nid)].extend(times)
return dict(merged)

def _compute_familiarity(self, ca3_active: Set[int]) -> Tuple[bool, int]
"""
Neural familiarity: compare current CA3 activity to stored engrams.
Returns (is_familiar, best_match_index).
"""
if not self.grams:
    return False, -1

best_jaccard = 0.0
best_idx = -1
for idx, engram in enumerate(self.grams):
    intersection = len(ca3_active & engram)
    union = len(ca3_active | engram)
    jaccard = intersection / union if union > 0 else 0.0
    if jaccard > best_jaccard:
        best_jaccard = jaccard
        best_idx = idx

# Threshold: if overlap > 20% with any engram, consider familiar
is_familiar = best_jaccard > 0.20
return is_familiar, best_idx

def encode_episode(self, sql_row: Dict, graph_edge: Tuple,
                  text: Optional[str] = None) -> Dict:
"""
Two-stage encoding with CONTINUOUS state.
Stage 1 (10ms): Pre-activation, M=0
Stage 2 (50ms): Plasticity, M=1 or 2
"""
# 1. Encode inputs
sql_spikes = self.sql_enc.encode(sql_row)
graph_spikes = self.graph_enc.encode(graph_edge)

# Convert to rate vectors for EC/DG
x_sql = np.zeros(100)
for nid, times in sql_spikes.items():
    x_sql[nid] = len(times)
x_graph = np.zeros(80)
for nid, times in graph_spikes.items():
    x_graph[nid] = len(times)

# Text
text_spikes = {}
if text and self.text_enc:
    text_spikes = self.text_enc.encode(text)
    x_text = np.zeros(100)
    for nid, times in text_spikes.items():
        x_text[nid] = len(times)
    x_ec = np.concatenate([x_sql, x_graph, x_text])

```

```

else:
    x_ec = np.concatenate([x_sql, x_graph])

# 2. EC novelty detection
energy, M_base = self.ec.compute_novelty(sql_spikes, graph_spikes,
                                         text_spikes if text else None)

# 3. DG sparse separation
dg_active = self.dg.encode(x_ec)
dg_spikes = self.dg.to_spikes(dg_active, t_offset=0.0)

# 4. Prepare unified input spikes for CA3
# SQL: "0".."99", Graph: "100".."179", Text: "180".."279", DG: "300"
all_inputs = self._merge_spikes(sql_spikes, graph_spikes, text_spikes)
for nid in dg_active:
    all_inputs[str(300 + nid)] = [1.0] # DG inputs

# 5. Stage 1: Pre-activation (10ms, M=0)
# CRITICAL FIX: No reset before this; fresh episode does reset
self.ca3.reset()
self.ca3.run(10.0, all_inputs, M=0.0)

# Check familiarity based on CA3 activity
ca3_active_pre = self.ca3.get_active_neurons(threshold=1)
is_familiar, match_idx = self._compute_familiarity(ca3_active_pre)

# Determine M for plasticity
if is_familiar:
    M = 1.0 # Reinforce
else:
    M = 2.0 # Novel: ACh boost

# 6. Stage 2: Plasticity (50ms, continuous state)
# CRITICAL FIX: No reset between stages. We continue from current state
self.ca3.run(50.0, all_inputs, M=M)

# Store engram
ca3_active = self.ca3.get_active_neurons(threshold=1)
if not is_familiar:
    self.engrams.append(ca3_active)
else:
    # Update stored engram to incorporate new activity
    self.engrams[match_idx] = ca3_active | self.engrams[match_idx]

self.episode_count += 1

return {
    'energy': energy,
    'M': M,
    'familiar': is_familiar,
    'ca3_active': ca3_active,
    'engram_id': match_idx if is_familiar else len(self.engrams) - 1
}

def retrieve(self, sql_cue: Optional[Dict] = None,
            graph_cue: Optional[Tuple] = None,

```

```

        text_cue: Optional[str] = None,
        duration: float = 50.0) -> Dict:
    """
    Retrieval with partial cue. No learning (M=0).
    Returns CA3 activity and CA1 reconstruction.
    """
    inputs = {}
    if sql_cue:
        spikes = self.sql_enc.encode(sql_cue)
        for nid, times in spikes.items():
            inputs[str(nid)] = times
    if graph_cue:
        spikes = self.graph_enc.encode(graph_cue)
        for nid, times in spikes.items():
            inputs[str(100 + nid)] = times
    if text_cue and self.text_enc:
        spikes = self.text_enc.encode(text_cue)
        for nid, times in spikes.items():
            inputs[str(180 + nid)] = times

    self.ca3.reset()
    ca3_spikes = self.ca3.run(duration, inputs, M=0.0)

    # CA1 reconstruction
    sql_recon = self.ca1.decode_sql(ca3_spikes, duration)
    graph_recon = self.ca1.decode_graph(ca3_spikes, duration)

    return {
        'ca3_active': self.ca3.get_active_neurons(threshold=1),
        'ca3_spikes': ca3_spikes,
        'sql_reconstruction': sql_recon,
        'graph_reconstruction': graph_recon,
    }

def compute_graph_retrieval_accuracy(self, retrieved: Dict,
                                     target_edge: Tuple[int, str, int])
    """
    Measure actual graph reconstruction quality.
    target_edge: (node_a, relation, node_b)
    """
    node_a, rel, node_b = target_edge
    ca3_active = retrieved['ca3_active']

    # Check if target node neurons are active
    src_neurons = self.graph_enc.get_active_neurons(node_a, 'source')
    tgt_neurons = self.graph_enc.get_active_neurons(node_b, 'target')

    src_retrieved = len(src_neurons & ca3_active)
    tgt_retrieved = len(tgt_neurons & ca3_active)

    # Heuristic: relation encoded by timing, so check if both sides active
    structure_acc = (src_retrieved > 0 and tgt_retrieved > 0)

    return {
        'src_neurons_active': src_retrieved,
        'tgt_neurons_active': tgt_retrieved,
    }

```

```

        'structure_accuracy': float(structure_acc),
        'ca3_overlap_pct': len(ca3_active) / self.ca3.n_e * 100,
    }

```

```

In [3]: # =====
# 9. TESTS
# =====

def run_tests():
    """Run specification tests."""
    print("=" * 60)
    print("POLYCHRONOUS MULTI-MODAL SPIKING MEMORY – TEST SUITE")
    print("=" * 60)

    rng = np.random.RandomState(42)

    print("\n[Test 1] Encoder Reconstructibility")
    sql_enc = SQLEncoder()
    row = {'age': 30, 'salary': 85000, 'city': 'NYC', 'role': 'Engineer', 'c
    spikes = sql_enc.encode(row)

    age_decoded = sql_enc.decode_numeric(spikes, 'age')
    salary_decoded = sql_enc.decode_numeric(spikes, 'salary')
    city_decoded = sql_enc.decode_category(spikes, 'city')

    age_err = abs(age_decoded - 30)
    salary_err = abs(salary_decoded - 85000)

    print(f" Age: {age_decoded:.1f} (error: {age_err:.1f}) {'PASS' if age_e
    print(f" Salary: {salary_decoded:.0f} (error: {salary_err:.0f}) {'PASS'
    print(f" City: {city_decoded} {'PASS' if city_decoded == 'NYC' else 'FA

    graph_enc = GraphEncoder()
    g_spikes = graph_enc.encode((0, 'WORKS_AT', 1))
    src_times = [t for nid, times in g_spikes.items() if nid < 20 for t in t
    tgt_times = [t for nid, times in g_spikes.items() if nid >= 20 for t in
    if src_times and tgt_times:
        delay = np.mean(tgt_times) - np.mean(src_times)
        print(f" Graph delay: {delay:.2f}ms {'PASS' if abs(delay - 3.0) < 1

    print("\n[Test 2] CA3 Pattern Completion")
    ca3 = CA3Attractor(seed=42)
    patterns = []
    for p in range(10):
        active = set(rng.choice(ca3.n_e, 15, replace=False))
        patterns.append(active)
        for i in active:
            for j in active:
                if i != j:
                    for syn in ca3.incoming[j]:
                        if syn.pre == str(i):
                            syn.w = min(syn.w + 2.0, syn.w_max)

    correct = 0
    overlaps = []
    for pattern in patterns:

```



```

    cue = set(rng.choice(list(pattern), 8, replace=False))
    ca3.reset()
    inputs = {str(n): [1.0] for n in cue}
    ca3.run(50.0, inputs, M=0.0)
    retrieved = ca3.get_active_neurons(threshold=1)

    best_jaccard = 0.0
    for p in patterns:
        jacc = len(retrieved & p) / len(retrieved | p) if (retrieved | p) else 0.0
        best_jaccard = max(best_jaccard, jacc)
    overlaps.append(best_jaccard)
    if best_jaccard > 0.5:
        correct += 1

print(f" Retrieval accuracy: {correct}/10 {'PASS' if correct >= 7 else 'FAIL'}")
print(f" Mean overlap: {np.mean(overlaps):.2f} {'PASS' if np.mean(overlaps) > 0.5 else 'FAIL'}")

print("\n[Test 3] Multi-Modal Binding")
mem = MultiModalMemory(seed=42)
alice_sql = {'age': 30, 'salary': 120000, 'city': 'SF', 'role': 'Engineer'}
alice_graph = (0, 'WORKS_AT', 1)
result = mem.encode_episode(alice_sql, alice_graph)
print(f" Learned engram {result['engram_id']}: {len(result['ca3_active'])} neurons")

retrieved = mem.retrieve(sql_cue=alice_sql)
random_sql = {'age': 55, 'salary': 40000, 'city': 'TX', 'role': 'Sales'}
random_retrieved = mem.retrieve(sql_cue=random_sql)

diff = len(retrieved['ca3_active']) - len(random_retrieved['ca3_active'])
print(f" Cue CA3 neurons: {len(retrieved['ca3_active'])}")
print(f" Random CA3 neurons: {len(random_retrieved['ca3_active'])}")
print(f" Difference: {diff} {'PASS' if diff > 2 else 'FAIL'}")

acc = mem.compute_graph_retrieval_accuracy(retrieved, alice_graph)
print(f" Graph structure accuracy: {acc['structure_accuracy']} "
      f"(src={acc['src_neurons_active']}, tgt={acc['tgt_neurons_active']})")

print("\n[Test 4] Engram Stability")
mem2 = MultiModalMemory(seed=123)
episode = ({'age': 25, 'salary': 90000, 'city': 'NYC', 'role': 'PM', 'department': 'Engineering', 'reports_to': 2},
           (2, 'MANAGES', 3))

engrams = []
for _ in range(10):
    res = mem2.encode_episode(*episode)
    engrams.append(res['ca3_active'])

jaccards = []
first = engrams[0]
for later in engrams[1:]:
    inter = len(first & later)
    union = len(first | later)
    jaccards.append(inter / union if union > 0 else 0.0)

mean_jacc = np.mean(jaccards)
print(f" Mean Jaccard overlap vs first: {mean_jacc:.2f} {'PASS' if mean_jacc > 0.5 else 'FAIL'}")

```

```

print("\n[Test 5] EC Novelty Detection")
ec = EntorhinalCortex()
sql1 = {i: [1.0, 2.0] for i in range(10)}
graph1 = {i: [1.0] for i in range(100, 110)}
e1, M1 = ec.compute_novelty(sql1, graph1, duration=10.0)
print(f"  First presentation: energy={e1:.1f}, M={M1} {'PASS' if M1 == 2

sql2 = {i: [1.0, 2.0, 2.5] for i in range(10)}
graph2 = {i: [1.0, 1.5] for i in range(100, 110)}
e2, M2 = ec.compute_novelty(sql2, graph2, duration=10.0)
print(f"  Second presentation: energy={e2:.1f}, M={M2} {'PASS' if M2 ==

print("\n[Test 6] Continuous Two-Stage Protocol")
mem3 = MultiModalMemory(seed=99)
ep = ({'age': 40, 'salary': 150000, 'city': 'LA', 'role': 'CEO', 'dept':
      (1, 'FRIENDS_WITH', 2)})

mem3.ca3.reset()
sql_spikes = mem3.sql_enc.encode(ep[0])
graph_spikes = mem3.graph_enc.encode(ep[1])
inputs = mem3._merge_spikes(sql_spikes, graph_spikes)

mem3.ca3.run(10.0, inputs, M=0.0)
v_after_pre = [n.v for n in mem3.ca3.neurons[:5]]
spikes_after_pre = [len(n.spike_times) for n in mem3.ca3.neurons]

mem3.ca3.run(50.0, inputs, M=2.0)
v_after_plastic = [n.v for n in mem3.ca3.neurons[:5]]

continuous = not all(v == -70.0 for v in v_after_plastic)
print(f"  Membrane potentials preserved: {continuous} {'PASS' if continu
print(f"  CA3 activity after pre: {sum(spikes_after_pre)} spikes")

print("\n" + "=" * 60)
print("TEST SUITE COMPLETE")
print("=" * 60)

if __name__ == "__main__":
    run_tests()

```

```
=====
POLYCHRONOUS MULTI-MODAL SPIKING MEMORY – TEST SUITE
=====
```

[Test 1] Encoder Reconstructibility

Age: 30.6 (error: 0.6) PASS
 Salary: 93609 (error: 8609) FAIL
 City: NYC PASS
 Graph delay: 3.00ms PASS

[Test 2] CA3 Pattern Completion

Retrieval accuracy: 0/10 FAIL
 Mean overlap: 0.00 FAIL

[Test 3] Multi-Modal Binding

Learned engram 0: 0 CA3 neurons (M=2.0)
 Cue CA3 neurons: 0
 Random CA3 neurons: 0
 Difference: 0 FAIL
 Graph structure accuracy: 0.0 (src=0, tgt=0)

[Test 4] Engram Stability

Mean Jaccard overlap vs first: 0.00 FAIL

[Test 5] EC Novelty Detection

First presentation: energy=400000.0, M=1 FAIL
 Second presentation: energy=900000.0, M=1 PASS

[Test 6] Continuous Two-Stage Protocol

Membrane potentials preserved: False FAIL
 CA3 activity after pre: 0 spikes

```
=====
TEST SUITE COMPLETE
=====
```

```
In [4]: # Quick demo
mem = MultiModalMemory(seed=42)
result = mem.encode_episode({'age': 30, 'salary': 120000, 'city': 'SF', 'rol
result
```

```
Out[4]: {'energy': np.float64(470000.0),
'M': 2.0,
'familiar': False,
'ca3_active': set(),
'engram_id': 0}
```